

Práctica 4: Desarrollo y orquestación de microservicios

En el panorama actual de la ingeniería de software, la adopción de arquitecturas nativas en nube ha convertido a herramientas como Docker y Kubernetes en estándares de la industria. Este documento constituye una guía teórico-práctica diseñada para acompañar al estudiante desde los fundamentos básicos de la contenerización hasta la orquestación avanzada de clústeres. A través de un enfoque progresivo, el estudiante aprenderá a empaquetar aplicaciones, gestionar su ciclo de vida y diseñar infraestructuras resilientes. El objetivo principal de esta guía es dotar al estudiante de las bases tecnológicas y metodológicas necesarias para afrontar con éxito el reto final del laboratorio: el despliegue integral de una arquitectura de microservicios moderna, escalable y segura.

1. Instalación de Docker

En esta sección se detallan los pasos necesarios para instalar **Docker** en el sistema operativo Linux Debian, siguiendo su documentación oficial [1]. **Docker** es compatible con la mayoría de sistemas operativos; en caso de requerir su instalación en un entorno distinto, se recomienda consultar dicha documentación.

Para llevar a cabo la instalación en Debian, es necesario configurar primero el repositorio APT y así utilizar el gestor de paquetes nativo de esta distribución. En primer lugar, añadiremos la clave GPG oficial de **Docker** y el repositorio a la lista de fuentes utilizando los siguientes comandos:

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /
    ↪ etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Una vez configurado el repositorio APT, procedemos a instalar los paquetes necesarios del motor de Docker. Para ello, utilizamos el siguiente comando:

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-
  ➔ buildx-plugin docker-compose-plugin
```

Por defecto, el servicio de Docker se inicia automáticamente una vez terminada la instalación. Podemos comprobar que se está ejecutando consultando el estado del servicio `docker` a través del comando `systemctl`. Una vez nos cercioramos de que el servicio está activo, podemos verificar su funcionamiento lanzando un contenedor de prueba con el siguiente comando:

```
sudo docker run --name miapache -p 8080:80 httpd
```

Este comando crea un contenedor con el nombre `miapache` basado en la imagen oficial `httpd` y mapea el puerto 8080 de nuestra máquina anfitriona (*host*) al puerto 80 del contenedor. Si el despliegue es exitoso, al acceder a través de nuestro navegador a la dirección `http://localhost:8080` deberíamos visualizar la página de inicio de Apache. Esto confirmará que Docker está instalado y operativo.

Por otro lado, el demonio de Docker se ejecuta siempre con privilegios de administrador (`root`), por tanto, para poder comunicarnos con él es necesario usar el cliente como usuario privilegiado (mediante `sudo`). Para poder utilizar nuestro usuario estándar sin necesidad de elevar privilegios en cada comando, debemos añadirlo al grupo `docker`. Para ello, ejecutaremos la siguiente secuencia:

```
sudo groupadd docker 2>/dev/null || true
sudo usermod -aG docker $USER
newgrp docker
```

Es importante destacar que incluir a un usuario en el grupo `docker` es equivalente a otorgarle privilegios de `root`, lo que puede conllevar riesgos de seguridad. La manera más segura de operar, especialmente en entornos de producción, es utilizar el modo `rootless`, tal y como indica la documentación oficial [2].

2. Primeros pasos con Docker

Docker es una plataforma de contenedorización de código abierto que permite empaquetar, distribuir y ejecutar aplicaciones de forma aislada mediante contenedores. A diferencia de las máquinas virtuales tradicionales, los contenedores no incluyen un sistema operativo completo, sino que comparten el núcleo del sistema anfitrión, lo que reduce significativamente el consumo de recursos y mejora los tiempos de arranque. Este modelo favorece arquitecturas basadas en microservicios y entornos reproducibles, garantizando consistencia entre desarrollo, pruebas y producción.

Desde el punto de vista técnico, **Docker** se apoya en funcionalidades nativas del núcleo de Linux como los *namespaces*, que proporcionan aislamiento de procesos, red y sistema de archivos; los *cgroups*, que permiten limitar y controlar el uso de recursos como CPU y memoria; y sistemas de archivos en capas como *OverlayFS*, que posibilitan la construcción eficiente de imágenes mediante capas inmutables. En versiones actuales, **Docker** utiliza *containerd* como runtime de contenedores y sigue el estándar *Open Container Initiative* (OCI), lo que garantiza compatibilidad con otros motores como CRI-O.

En conjunto, **Docker** constituye una tecnología fundamental dentro de los procesos modernos de integración y despliegue continuo, permitiendo construir artefactos reproducibles, versionables y fácilmente distribuibles, alineados con prácticas DevSecOps y despliegues basados en infraestructura como código.

2.1. Arquitectura general de Docker

La arquitectura de **Docker** se compone principalmente del cliente **Docker**, el demonio *dockerd* y un registro de imágenes. El cliente se comunica con el demonio a través de una API REST, permitiendo construir imágenes, crear contenedores y gestionar redes y volúmenes. La Figura 1 muestra la arquitectura conceptual detallada del sistema.

2.2. Imágenes y el archivo «Dockerfile»

Una imagen **Docker** es una plantilla inmutable y de solo lectura que contiene todo lo necesario para ejecutar una aplicación, incluyendo código, dependencias, bibliotecas y configuración. Las imágenes se construyen en capas, donde cada instrucción del proceso de construcción genera una nueva capa, lo que permite la reutilización y optimización del almacenamiento. Las prácticas actuales recomiendan el uso de imágenes base mínimas, como Alpine o distribuciones *distroless*, así como la construcción multifase (*multi-stage builds*) para reducir el tamaño final y minimizar la superficie de ataque.

La creación de imágenes se define mediante un archivo denominado «**Dockerfile**», que contiene instrucciones declarativas. Las más comunes son:

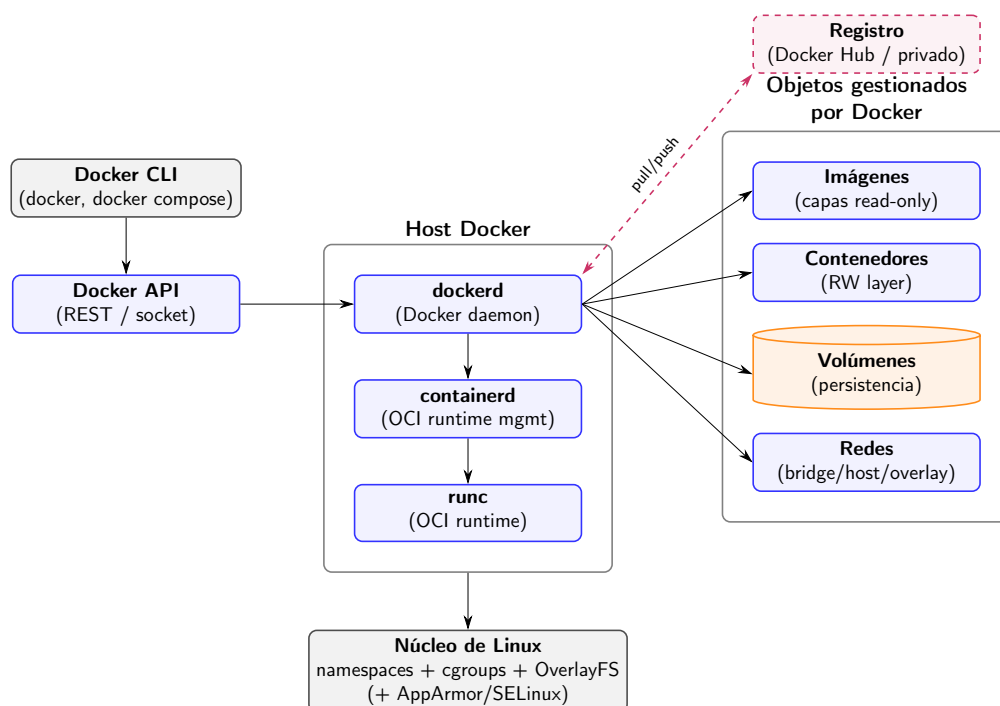


Figura 1: Arquitectura interna de Docker y flujo de ejecución.

- **FROM**: Especifica la imagen base de la que se parte.
- **RUN**: Ejecuta comandos en una nueva capa durante la construcción.
- **COPY** y **ADD**: Incorporan archivos o directorios del host al contenedor.
- **WORKDIR**: Define el directorio de trabajo para las siguientes instrucciones.
- **ENV**: Establece variables de entorno.
- **EXPOSE**: Documenta los puertos en los que escucha el contenedor.
- **ENTRYPOINT** y **CMD**: Determinan el comando y los parámetros por defecto del proceso principal del contenedor.
- **HEALTHCHECK**: Permite definir verificaciones de estado de la aplicación.

En la actualidad, la directiva **MAINTAINER** está obsoleta y se sustituye por etiquetas mediante la instrucción **LABEL**. También es altamente recomendable el uso del fichero **.dockerignore** para evitar enviar archivos innecesarios al contexto de construcción del demonio.

Ejemplo práctico: construcción de la imagen

Para ilustrar estos conceptos, vamos a empaquetar una aplicación web sencilla. Supongamos que tenemos un pequeño script en Python llamado `hola_mundo.py` que actuará como nuestra aplicación web (utilizando la interfaz CGI):

```
#!/usr/bin/env python3
print("Content-type: text/html\n")
print("<h1>Hola Mundo desde Docker con Apache y Python!</h1>")
```

Para *dockerizar* esta aplicación, crearemos un archivo llamado «Dockerfile» (**sin extensión**) en el mismo directorio que nuestro script, con el siguiente contenido:

```
# 1. Definimos la imagen base
FROM debian:12-slim

# 2. Añadimos metadatos (etiquetas)
LABEL version="1.0"
LABEL description="Servidor Apache con soporte Python CGI"

# 3. Instalamos Apache y Python, limpiando la caché de apt
RUN apt-get update && \
    apt-get install -y apache2 python3 && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

# 4. Habilitamos el módulo CGI de Apache
RUN a2enmod cgi

# 5. Copiamos nuestra aplicación al directorio CGI de Apache
COPY hola_mundo.py /usr/lib/cgi-bin/hola_mundo.py

# 6. Otorgamos permisos de ejecución al script
RUN chmod +x /usr/lib/cgi-bin/hola_mundo.py

# 7. Documentamos que el contenedor escuchará en el puerto 80
EXPOSE 80

# 8. Definimos el comando principal
CMD ["apachectl", "-D", "FOREGROUND"]
```

Una vez definidos ambos archivos, el cliente de **Docker** enviará este *contexto* al demonio para construir la imagen. Para ello, ejecutamos el comando de construcción (**build**) en la terminal, asignándole una etiqueta o nombre (**-t**) y especificando el directorio actual (**.**):

```
sudo docker build -t mi-apache-python:1.0 .
```

Al finalizar este comando, tendremos nuestra imagen inmutable lista y almacenada en el registro local de nuestra máquina, a la espera de ser instanciada.

2.3. Contenedores: Instancias en ejecución

Un contenedor es una instancia en ejecución de una imagen. Cuando se crea un contenedor, **Docker** añade una capa superior de lectura y escritura (RW) sobre la imagen base de solo lectura. Los cambios realizados en esa capa desaparecen al eliminar el contenedor, lo que refuerza el principio de inmutabilidad.

Conceptualmente, un contenedor debe ejecutar un único proceso principal, aunque puede gestionar procesos secundarios internos si la aplicación lo requiere. Esta filosofía simplifica la observabilidad, el control de logs y la escalabilidad horizontal.

Ejemplo práctico: despliegue del contenedor

Retomando el ejemplo anterior, una vez que ya disponemos de la imagen `mi-apache-python:1.0`, podemos instanciar un contenedor a partir de ella. El siguiente comando arranca el contenedor en segundo plano (`-d`) y mapea el puerto 8080 de nuestra máquina física al puerto 80 aislado del contenedor:

```
sudo docker run -d -p 8080:80 --name servidor-web mi-apache-python  
↪ :1.0
```

Si el despliegue es correcto, el demonio habrá creado la capa de lectura/escritura y el contenedor estará ejecutando **Apache**. Podemos probar nuestra aplicación abriendo un navegador web o utilizando el comando `curl` desde la terminal de nuestro *host*:

```
curl http://localhost:8080/cgi-bin/hola_mundo.py
```

Para detener la ejecución del contenedor en cualquier momento, emplearemos el comando `stop` indicando su nombre:

```
sudo docker stop servidor-web
```

Cabe destacar que, si durante la creación no se especifica la opción `-name`, **Docker** asignará automáticamente un nombre aleatorio al contenedor. Para listar los contenedores existentes en el sistema (tanto los que están en ejecución como los actualmente parados), se utiliza el comando `docker ps -all` (o su forma abreviada `-a`). A través de la salida de este comando, podemos obtener el identificador único (ID) o el nombre del contenedor para gestionarlo.

Finalmente, si deseamos destruir el contenedor de forma permanente y liberar su capa de lectura/escritura, utilizaremos el comando `rm` en lugar de `stop`. Por motivos de se-

guridad, es necesario que el contenedor haya sido detenido previamente antes de poder eliminarlo.

```
sudo docker rm servidor-web
```

2.4. Persistencia de datos: volúmenes

Dado que los contenedores son efímeros, **Docker** proporciona varios mecanismos para la persistencia de datos y configuración:

- **Volúmenes (Volumes):** Son gestionados por **Docker** y se almacenan en una ruta específica administrada por el motor. Son el mecanismo recomendado para entornos productivos por su facilidad de respaldo, control de acceso y aislamiento del host.
- **Bind mounts:** Permiten montar un directorio o archivo específico del sistema anfitrión directamente dentro del contenedor. Son útiles en desarrollo para reflejar cambios de código en tiempo real.
- **Sistemas en memoria (*tmpfs*):** Permiten un almacenamiento efímero directamente en la memoria RAM del host, sin escritura en disco. Útiles para guardar datos sensibles o temporales.

Ejemplo práctico: creación y uso de un volumen

Para ilustrar el uso de volúmenes gestionados, vamos a crear uno explícitamente y asociarlo a un contenedor de base de datos (por ejemplo, PostgreSQL). De este modo, garantizamos que los datos sobrevivan aunque el contenedor se detenga o sea eliminado. Para una información más detallada, se recomienda consultar la documentación oficial de almacenamiento en **Docker** [3].

```
# 1. Crear un volumen gestionado por Docker
sudo docker volume create mis_datos_pg

# 2. Arrancar un contenedor de PostgreSQL montando el volumen
# El flag -v enlaza el volumen local con la ruta de datos del
  ↪ contenedor
sudo docker run -d --name base_datos -e POSTGRES_PASSWORD=secreto
  ↪ -v mis_datos_pg:/var/lib/postgresql/data postgres:14
```

2.5. Comunicaciones y redes

Docker incorpora un subsistema de redes que permite la comunicación entre contenedores y con el exterior, ofreciendo distintos modos de configuración:

- **Bridge:** Red por defecto. Crea un puente virtual para la comunicación interna entre contenedores ubicados en el mismo host.
- **Host:** Elimina el aislamiento de red entre el contenedor y el anfitrión, compartiendo directamente la pila de red del sistema operativo base.
- **Overlay:** Diseñada para despliegues distribuidos, permite la comunicación segura entre contenedores ubicados en distintos nodos (generalmente usando **Docker Swarm**).
- **None:** Desactiva por completo la conectividad de red del contenedor.

Las redes *bridge* definidas por el usuario ofrecen la ventaja de proporcionar resolución DNS automática entre servicios utilizando el nombre del contenedor, lo que facilita enormemente el diseño de arquitecturas desacopladas. Para profundizar en las capacidades de red y configuraciones avanzadas, se puede consultar la documentación oficial de redes de Docker [4].

Ejemplo práctico: redes y resolución DNS

A continuación, crearemos una red *bridge* personalizada y desplegaremos dos contenedores en ella para demostrar cómo pueden comunicarse utilizando únicamente sus nombres, gracias al servidor DNS integrado de Docker:

```
# 1. Crear una red de tipo bridge definida por el usuario
sudo docker network create mi_red_interna

# 2. Arrancar un contenedor conectado a esta red
sudo docker run -d --name mi_bd --network mi_red_interna -e
    ↪ POSTGRES_PASSWORD=secreto postgres:14

# 3. Arrancar un contenedor temporal interactivo en la misma red
# y comprobar la conectividad haciendo ping al nombre 'mi_bd'
sudo docker run -it --rm --network mi_red_interna alpine ping -c 4
    ↪ mi_bd
```

2.6. Registros de imágenes y Docker Hub

Las imágenes construidas deben almacenarse y distribuirse. Para ello se utilizan los registros (*registries*). El cliente interactúa con estos registros mediante comandos como **push** (para subir una imagen) o **pull** (para descargarla explícitamente).

El registro público por defecto es **Docker Hub** [5], que aloja miles de imágenes oficiales de bases de datos, lenguajes y servicios. No obstante, en entornos corporativos es estándar

el uso de registros privados (como GitLab Container Registry, AWS ECR o Harbor) para garantizar la seguridad y privacidad del software propietario.

Cabe destacar que cada vez que lanzamos una instancia de un contenedor con el comando `docker run`, el motor busca primero si la imagen solicitada se encuentra en el caché local. Si no la encuentra, intenta descargarla automáticamente del registro remoto configurado (por defecto, `Docker Hub`). Este mismo comportamiento ocurre durante la construcción de imágenes al utilizar la directiva `FROM` dentro del fichero «`Dockerfile`».

2.7. Automatización con Docker Compose

Para la gestión de aplicaciones compuestas por múltiples servicios (como un backend, un frontend y una base de datos), `Docker` proporciona `Docker Compose`. Esta herramienta permite definir de forma declarativa la configuración de varios contenedores en un único archivo YAML (`docker-compose.yml` o `compose.yaml`).

En versiones actuales, `Compose` se integra como un subcomando nativo (`docker compose`) en lugar del antiguo binario independiente (`docker-compose`). A través de este archivo es posible definir de un vistazo los servicios, redes, volúmenes, mapeos de puertos, variables de entorno y el orden de arranque (*dependencias*) de los componentes.

Aunque `Compose` no sustituye a un orquestador de producción como `Kubernetes`, resulta una herramienta invaluable y estandarizada para levantar entornos completos de desarrollo, pruebas y laboratorios formativos con un solo comando.

Ejemplo práctico: despliegue multicontenedor

Para ilustrar su utilidad, supongamos que deseamos desplegar un gestor de contenidos `WordPress`, el cual requiere tanto el servidor web con `PHP` como una base de datos `MariaDB`. En lugar de ejecutar múltiples comandos largos en la terminal y configurar redes manualmente, podemos definir toda la infraestructura en un archivo `docker-compose.yml`:

```
services:
  db:
    image: mariadb:10.11
    environment:
      MYSQL_ROOT_PASSWORD: password_root
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wp_user
      MYSQL_PASSWORD: wp_password
    volumes:
      - wp_data:/var/lib/mysql
    restart: always
```

```
wordpress:
  image: wordpress:latest
  ports:
    - "8080:80"
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_USER: wp_user
    WORDPRESS_DB_PASSWORD: wp_password
    WORDPRESS_DB_NAME: wordpress
  depends_on:
    - db
  restart: always

volumes:
  wp_data:
```

En este archivo se definen dos servicios (**db** y **wordpress**), se mapean los puertos para el acceso web desde el exterior y se crea un volumen persistente explícito (**wp_data**) para evitar que la base de datos se pierda. Además, Compose crea automáticamente una red interna, lo que permite que el contenedor de **WordPress** se conecte a la base de datos indicando simplemente el nombre del servicio (**WORDPRESS_DB_HOST: db**).

Para levantar toda esta infraestructura en segundo plano, basta con situarse en el directorio donde se encuentra el archivo YAML y ejecutar:

```
sudo docker compose up -d
```

Si el proceso finaliza con éxito, WordPress estará disponible en **http://localhost:8080**. Una vez que hayamos terminado nuestro trabajo o pruebas, podemos detener y eliminar los contenedores y la red creada por Compose de forma limpia ejecutando el siguiente comando:

```
sudo docker compose down
```

3. Orquestación con Kubernetes

Mientras que **Docker** y **Docker Compose** son herramientas excelentes para el desarrollo y despliegue de aplicaciones en un único servidor, en entornos productivos de alta disponibilidad surge la necesidad de gestionar múltiples servidores (un clúster), equilibrar la carga, escalar servicios automáticamente y recuperar el sistema frente a fallos. Para resolver estos problemas de *orquestación*, la herramienta estándar en la industria es **Kubernetes** (a menudo abreviado como **K8s**).

Para el desarrollo local y el aprendizaje de esta tecnología, utilizaremos **Minikube**, una

utilidad que despliega un clúster de Kubernetes de un solo nodo en nuestra máquina local.

3.1. Requisitos e instalación de Minikube

Antes de proceder con la instalación, es necesario asegurar que el equipo anfitrión cumple con los siguientes **requisitos mínimos**:

- 2 CPUs o más.
- 2 GB de memoria RAM libre.
- 20 GB de espacio libre en disco.
- Conexión a Internet.
- Un gestor de contenedores o máquinas virtuales instalado (en nuestro caso, **Docker**).

A continuación se detallan los pasos de instalación para el sistema operativo Linux Debian. Si se utiliza otro entorno (Windows o macOS), se debe consultar la documentación oficial de Minikube [6].

La interacción con el clúster de Kubernetes no se realiza de forma directa, sino mediante la herramienta de línea de comandos `kubectl`. Procederemos a descargar e instalar tanto el cliente como el propio Minikube:

```
# 1. Instalar el cliente kubectl
sudo apt-get install kubectl

# 2. Descargar e instalar Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/
    ↪ minikube_latest_amd64.deb
sudo dpkg -i minikube_latest_amd64.deb
```

Una vez instalados, podemos iniciar nuestro clúster local indicando que utilice **Docker** como motor subyacente:

```
minikube start --driver=docker
```

3.2. Arquitectura general de Kubernetes

Un clúster de Kubernetes se divide fundamentalmente en dos planos: el **Plano de Control** (*Control Plane* o nodo *Master*) y los **Nodos de Trabajo** (*Worker Nodes*) [7]. La Figura 2 ilustra esta topología.

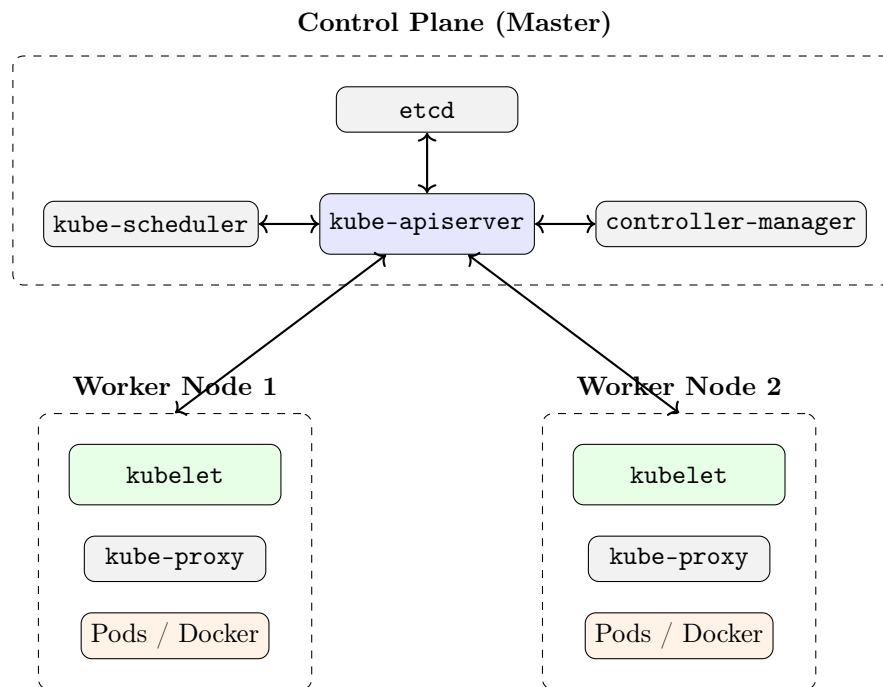


Figura 2: Arquitectura simplificada de un clúster de Kubernetes.

El **Control Plane** toma las decisiones globales sobre el clúster (como la planificación o la respuesta a eventos). Sus componentes principales son el **kube-apiserver** (el punto de entrada central), **etcd** (almacén de clave-valor con el estado del clúster), el **kube-scheduler** (asigna recursos) y el **kube-controller-manager** (mantiene el estado deseado).

Por su parte, los **Nodos** son las máquinas donde se ejecutan realmente las cargas de trabajo. Cada nodo cuenta con un **kubelet** (el agente que se comunica con el Master) y un **kube-proxy** (gestiona las reglas de red).

3.3. Pods

En **Docker**, la unidad mínima de ejecución es el *contenedor*. Sin embargo, **Kubernetes** **no ejecuta contenedores directamente, sino Pods**.

Un **Pod** es la unidad de despliegue más pequeña y atómica en **Kubernetes**. Actúa como un envoltorio lógico que encapsula uno o más contenedores, recursos de almacenamiento compartidos (volúmenes) y una IP de red única.

Aunque el patrón de diseño más habitual es el de *un contenedor por Pod*, existen arquitecturas avanzadas donde un Pod agrupa un contenedor principal (por ejemplo, servidor

web) y un contenedor secundario auxiliar (por ejemplo, un recolector de métricas). Al estar en el mismo Pod, ambos contenedores comparten el mismo ciclo de vida, se ejecutan en el mismo nodo físico y pueden comunicarse a través de `localhost`.

3.4. Interacción con el clúster: `kubectl`

A diferencia de **Docker**, donde priman los comandos imperativos (por ejemplo, “*ejecuta este contenedor*”), **Kubernetes** adopta un enfoque **declarativo**. El usuario redacta un archivo YAML describiendo el *estado deseado* de la infraestructura, y el Plano de Control se encarga de hacerlo realidad.

Los comandos de `kubectl` más utilizados para el día a día son:

- `kubectl apply -f archivo.yaml`: Crea o actualiza los recursos definidos en el archivo YAML especificado.
- `kubectl get <recurso>`: Lista los objetos instanciados (por ejemplo, `kubectl get pods`).
- `kubectl describe <recurso> <nombre>`: Muestra información detallada y eventos de un objeto concreto, siendo vital para la depuración.
- `kubectl logs <nombre_pod>`: Muestra la salida estándar del contenedor alojado en el Pod.
- `kubectl delete -f archivo.yaml`: Elimina los recursos instanciados por ese archivo.

3.5. Definición de objetos mediante YAML

Todo objeto definido en **Kubernetes** comparte una estructura base de cuatro campos: `apiVersion`, `kind` (tipo de objeto), `metadata` (identificación como nombre y etiquetas) y `spec` (especificación del estado). La plataforma ofrece una inmensa cantidad de opciones de configuración para cada objeto. Para profundizar en las características avanzadas de los *Services*, *Deployments*, *Ingress* y el resto de recursos, se recomienda encarecidamente consultar la documentación oficial disponible en <https://kubernetes.io/docs/home/>.

A continuación se detallan los objetos fundamentales para desplegar aplicaciones:

Deployments y ReplicaSets

Un **Deployment** gestiona la creación de Pods y asegura que un número específico de ellos (gestionado internamente por un *ReplicaSet*) esté siempre en ejecución. Permite

actualizaciones sin cortes (llamado *rolling updates*).

Para que nuestro posterior ejemplo de Ingress funcione de manera visual, vamos a desplegar una primera aplicación (**app1**) con 2 réplicas. Utilizaremos una imagen de Nginx, pero modificaremos su arranque para que su página principal muestre que es la “Aplicación 1” y el identificador (nombre/IP) del Pod que está atendiendo la petición. Esto nos permitirá comprobar visualmente el balanceo de carga. El Código 1 muestra el código de ejemplo de esta aplicación.

Código 1: Despliegue de la Aplicación 1 («app1-deployment.yaml»).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-app1
spec:
  replicas: 2                # 2 copias exactas para ver el balanceo
  selector:
    matchLabels:
      app: app1
  template:                  # Plantilla del Pod
    metadata:
      labels:
        app: app1
    spec:
      containers:
        - name: web
          image: nginx:alpine
          # Sobrescribimos el index.html para mostrar el nombre del
          ↪ Pod y luego arrancamos nginx
          command: ["/bin/sh", "-c"]
          args:
            - "echo '<h1>Soy la Aplicacion 1</h1><p>Atendiendo desde
            ↪ el Pod: '$\bd{hostname}'</p>' > /usr/share/nginx/html/index
            ↪ .html && nginx -g 'daemon off;'"
          ports:
            - containerPort: 80
```

*Nota: Para crear la **app2**, bastaría con duplicar este archivo, cambiando las referencias de **app1** a **app2** y el mensaje del **echo** a “Soy la Aplicacion 2”.*

Services

Dado que los Pods son efímeros y sus direcciones IP cambian cada vez que son recreados (por ejemplo, si un nodo falla y el ReplicaSet levanta el Pod en otro lugar), un **Service** soluciona este problema proporcionando una IP estática interna y un nombre de dominio estable para un conjunto de Pods, identificándolos mediante sus etiquetas (*labels*).

Existen diferentes tipos de servicios dependiendo de si queremos que la aplicación sea accesible solamente desde dentro del clúster o también desde el exterior. El tipo se especifica en el campo `spec.type` del archivo YAML:

- **ClusterIP (por defecto):** Expone el servicio en una IP interna del clúster. Esto hace que el servicio **sólo sea accesible desde otros Pods dentro del mismo clúster**. Es la configuración ideal para bases de datos, cachés, o como en nuestro caso, para aplicaciones internas que serán expuestas de forma controlada a través de un Ingress.
- **NodePort:** Expone el servicio en la IP de cada Nodo en un puerto estático (por defecto en el rango 30000-32767). Permite el acceso externo directo al servicio redirigiendo el tráfico hacia el puerto del nodo.
- **LoadBalancer:** Expone el servicio externamente utilizando el balanceador de carga del proveedor de la nube subyacente (o el simulado por Minikube en nuestro entorno local).

Dado que el acceso desde el exterior lo gestionará más adelante el objeto Ingress, nuestros servicios internos de prueba sólo necesitan ser accesibles dentro del propio clúster. Por ello, utilizaremos el tipo `ClusterIP`. Configuraremos el puerto para que coincida con el que espera el Ingress (7777 para `app1` y 8888 para `app2`; véase ejemplo en Código 2).

Código 2: Servicio para la Aplicación 1 («`app1-service.yaml`»).

```
apiVersion: v1
kind: Service
metadata:
  name: app1          # Este nombre debe coincidir con el backend
                      ↪ del Ingress
spec:
  type: ClusterIP     # Accesible sólo desde dentro del clúster (
                      ↪ el Ingress lo consumirá)
  selector:
    app: app1         # Enruta el tráfico a los Pods con esta
                      ↪ etiqueta
  ports:
    - protocol: TCP
      port: 7777       # Puerto expuesto por el Service (el que
                      ↪ busca el Ingress)
      targetPort: 80   # Puerto interno donde escucha el
                      ↪ contenedor Nginx
```

De igual forma, para la segunda aplicación crearíamos un archivo similar donde el nombre fuera `app2`, el selector `app: app2` y el puerto expuesto fuera el 8888.

Secrets y ConfigMaps

Un **ConfigMap** inyecta configuraciones y variables de entorno no confidenciales, mientras que un **Secret** almacena datos sensibles (como contraseñas) codificados en Base64, evitando así *hardcodear* credenciales en el código fuente.

Es importante destacar que, aunque el objeto se denomine *Secret*, por defecto el contenido de estas variables se almacena únicamente codificado en Base64 (lo cual es un sistema de codificación, no de cifrado). Para proteger verdaderamente estos datos y cifrarlos en reposo dentro del almacén **etcd**, se deben aplicar configuraciones adicionales que pueden consultarse en la documentación oficial [8]. Se muestra un ejemplo en el Código 3.

Código 3: Ejemplo de un objeto Secret con credenciales codificadas en Base64.

```
apiVersion: v1
kind: Secret
metadata:
  name: credenciales-bd
type: Opaque
data:
  # Valores codificados previamente en base64
  usuario: cm9vdA==
  clave: c2Vjc mV0bzEyMw==
```

Almacenamiento y persistencia (PersistentVolumeClaims)

Como hemos mencionado anteriormente, los Pods en **Kubernetes** son efímeros. Esto significa que cualquier dato escrito en el sistema de archivos del contenedor se perderá irrevocablemente si el Pod se reinicia, falla o es reprogramado en otro nodo. Para aplicaciones sin estado (*stateless*) como el API Gateway o un servicio web simple, esto no supone un problema. Sin embargo, para componentes con estado (*stateful*) como las bases de datos (**MariaDB** o **PostgreSQL**), la persistencia de los datos es estrictamente necesaria.

Kubernetes resuelve este problema desacoplando el almacenamiento físico de los Pods mediante dos objetos principales:

- **PersistentVolume (PV):** Es una porción de almacenamiento real en el clúster (como un disco en la nube, almacenamiento en red NFS, o un directorio local en Minikube) provisionado por un administrador.
- **PersistentVolumeClaim (PVC):** Es una solicitud de almacenamiento realizada por un usuario (o desarrollador). El PVC reclama un PV que cumpla con sus requisitos de tamaño y modo de acceso.

Al definir un PVC, uno de los parámetros más importantes es el modo de acceso

(`accessModes`), que define cómo puede ser montado el volumen:

- **ReadWriteOnce (RWO):** El volumen puede ser montado como lectura y escritura por un único nodo. Es el modo más común para bases de datos relacionales.
- **ReadOnlyMany (ROX):** El volumen puede ser montado como solo lectura por múltiples nodos.
- **ReadWriteMany (RWX):** El volumen puede ser montado como lectura y escritura por múltiples nodos (muy útil para servidores de archivos compartidos).

En entornos de desarrollo local como **Minikube**, existe un *provisionador dinámico* activado por defecto. Esto significa que al crear un PVC, el clúster se encarga de crear automáticamente el PV subyacente sin necesidad de intervención manual. El Código 4 muestra cómo solicitar un volumen persistente de 1 Gigabyte.

Código 4: Ejemplo de solicitud de volumen persistente (PVC).

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mariadb-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Una vez creado el PVC, debemos indicarle a nuestro **Deployment** que lo utilice y lo monte en la ruta específica donde la aplicación guarda sus datos críticos (por ejemplo, `/var/lib/mysql` en el caso de **MariaDB**). El Código 5 ilustra este proceso, integrando además el uso de **Secrets** para proteger la contraseña de la base de datos.

Código 5: Montaje de un PVC y un Secret dentro de un Deployment.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mariadb
  template:
    metadata:
      labels:
        app: mariadb
    spec:
```

```

containers:
- name: mariadb
  image: mariadb:10.5
  env:
    - name: MYSQL_ROOT_PASSWORD
      valueFrom:
        secretKeyRef:
          name: credenciales-bd # Referencia al Secret
    ↪ definido anteriormente
      key: clave
    volumeMounts:
      - name: almacenamiento-bd # Referencia al nombre
    ↪ del volumen definido abajo
        mountPath: /var/lib/mysql # Ruta interna del
    ↪ contenedor a persistir
    volumes:
      - name: almacenamiento-bd
        persistentVolumeClaim:
          claimName: mariadb-pvc # Nombre exacto del PVC
    ↪ creado en el paso anterior

```

Ingress

Mientras que un *Service* opera a nivel de red (Capa 4), un **Ingress** opera a nivel de aplicación (Capa 7). Actúa como un proxy inverso y enrutador inteligente que permite exponer múltiples servicios internos a través de una única IP externa basándose en la ruta URL (por ejemplo, dirigir `/api` al backend y `/` al frontend).

En esta sección vamos a configurar un objeto Ingress que nos permitirá gestionar el acceso externo a los servicios de nuestro clúster. Ingress expone rutas HTTP/HTTPS basándose en las reglas que definamos. En primer lugar, es necesario activar el complemento (*addon*) de Ingress en el clúster de Minikube:

```
minikube addons enable ingress
```

A continuación, crearemos el archivo YAML del objeto Ingress y definiremos sus reglas. El objetivo es que, dependiendo de la ruta a la que acceda el usuario, el tráfico se redirija a una aplicación u otra. Por ejemplo:

- `http://mis-aplicaciones.com/app1` → Servicio app1
- `http://mis-aplicaciones.com/app2` → Servicio app2

Podemos definir reglas similares a como lo haríamos con los *VirtualHosts* de Apache o los bloques *server* de Nginx, de manera que cuando coincida la ruta, se encamine el tráfico hacia el servicio correspondiente. El Código 6 muestra el archivo de configuración

propuesto.

Código 6: Archivo de configuración del objeto Ingress.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mis-aplicaciones
spec:
  rules:
    - host: mis-aplicaciones-vulnerables.com
      http:
        paths:
          - path: /app1
            pathType: Prefix
            backend:
              service:
                name: app1
                port:
                  number: 7777
          - path: /app2
            pathType: Prefix
            backend:
              service:
                name: app2
                port:
                  number: 8888
```

Nota importante: *Debéis aseguraros de modificar el nombre y el número de puerto de los servicios en la sección **backend** para que coincidan con los nombres y puertos que hayáis utilizado en vuestros propios despliegues (**Deployments** y **Services**).*

Finalmente, instanciamos el objeto Ingress utilizando el comando:

```
kubectl apply -f ingress.yaml
```

Para comprobar que se ha creado correctamente y listar sus características, podemos utilizar el comando `kubectl get ingress`. En la salida de este comando, deberíais poder identificar la dirección IP pública asignada por Minikube al Ingress.

Para probar que el enrutamiento funciona correctamente desde nuestro navegador, y dado que no disponemos de un servidor DNS real que resuelva el dominio `mis-aplicaciones.com`, debemos engañar a nuestro sistema operativo. Para ello, editaremos el archivo de resolución local `/etc/hosts` (requiere permisos de administrador) y añadiremos una nueva línea con la IP que nos devolvió el comando anterior:

```
# Añadir al final del archivo /etc/hosts (sustituir la IP por la
  ↪ vuestra)
192.168.49.2  mis-aplicaciones.com
```

Una vez añadida esta línea, nuestro sistema operativo realizará la traducción del nombre de dominio a la dirección IP de Minikube localmente. Por tanto, si abrimos el navegador web:

- Al acceder a `http://mis-aplicaciones.com/app1`, el Ingress nos dirigirá al contenedor de la aplicación 1.
- Al acceder a `http://mis-aplicaciones.com/app2`, el Ingress nos dirigirá al contenedor de la aplicación 2.

4. Desarrollo y despliegue de una API REST

Para consolidar los conocimientos adquiridos sobre Docker y Kubernetes, vamos a realizar el ciclo de vida completo de una aplicación: desarrollaremos una pequeña API REST desde cero, la empaquetaremos en una imagen Docker y, finalmente, la desplegaremos en nuestro clúster local de Minikube utilizando nuestra propia imagen.

4.1. Desarrollo de la API REST

Una API REST (*REpresentational State Transfer*) es una interfaz que permite que diferentes sistemas se comuniquen a través del protocolo HTTP intercambiando datos, habitualmente en formato JSON.

Para nuestro ejemplo utilizaremos el lenguaje Python junto con el *framework* moderno **FastAPI**. Este entorno destaca por su altísimo rendimiento, su soporte nativo para programación asíncrona y la autogeneración de documentación. Crearemos un archivo llamado «main.py» que definirá dos rutas (URIs), tal y como se muestra en el Código 7.

Código 7: Código de la API REST («main.py»).

```
from fastapi import FastAPI
import uvicorn

app = FastAPI()

# Base de datos simulada en memoria
usuarios = [
    {"id": 1, "nombre": "Alice", "rol": "admin"},
    {"id": 2, "nombre": "Bob", "rol": "usuario"}
]

@app.get('/')
def home():
    return {"mensaje": "Bienvenido a mi API REST en Kubernetes"}
```

```
@app.get('/api/usuarios')
def get_usuarios():
    return {"usuarios": usuarios}

if __name__ == '__main__':
    # Uvicorn es el servidor web ASGI que ejecuta FastAPI
    uvicorn.run(app, host='0.0.0.0', port=8000)
```

4.2. Contenerización con Docker

El siguiente paso es empaquetar nuestra aplicación con todas sus dependencias para que pueda ejecutarse de manera aislada. Para ello, en el mismo directorio donde tenemos el archivo «main.py», crearemos un fichero llamado «Dockerfile» con el contenido mostrado en el Código 8.

Código 8: Archivo Dockerfile para empaquetar la API.

```
# Utilizamos una imagen base de Python ligera
FROM python:3.9-slim

# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# Instalamos FastAPI y el servidor Uvicorn
RUN pip install --no-cache-dir fastapi uvicorn

# Copiamos nuestro código fuente al contenedor
COPY main.py .

# Exponemos el puerto que usa Uvicorn
EXPOSE 8000

# Comando por defecto para ejecutar la API
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Ahora, construimos la imagen de Docker en nuestra máquina local asignándole la etiqueta `mi-api-rest:v1`. Posteriormente, podemos probarla localmente para asegurar que funciona:

```
# 1. Construir la imagen
docker build -t mi-api-rest:v1 .

# 2. Probar el contenedor localmente mapeando el puerto 8000
docker run -d -p 8000:8000 --name prueba-api mi-api-rest:v1

# 3. Comprobar con curl que responde correctamente
curl http://localhost:8000/api/usuarios
```

```
# 4. Detener y eliminar el contenedor de prueba
docker stop prueba-api
docker rm prueba-api
```

4.3. Despliegue en Kubernetes con imágenes locales

Al intentar desplegar un Pod en Minikube, el comportamiento por defecto de Kubernetes es buscar la imagen en un registro público (como Docker Hub). Si intentamos usar nuestra imagen `mi-api-rest:v1`, el clúster fallará con un error *ImagePullBackOff* porque la imagen sólo existe en nuestro entorno local.

Para solucionar esto, disponemos del comando `minikube image load`, que transfiere una imagen desde nuestro demonio Docker al clúster de Minikube:

```
# Cargar la imagen local en Minikube
minikube image load mi-api-rest:v1
```

Una vez cargada, redactaremos nuestro archivo manifiesto («`api-k8s.yaml`») como el que se muestra en el Código 9. Es fundamental añadir la directiva `imagePullPolicy: Never` en la especificación del contenedor para obligar a Kubernetes a utilizar la imagen local que acabamos de cargar.

Código 9: Despliegue y Servicio de la API («`api-k8s.yaml`»).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api-rest
  template:
    metadata:
      labels:
        app: api-rest
    spec:
      containers:
        - name: backend-api
          image: mi-api-rest:v1
          imagePullPolicy: Never # Usa la imagen local de Minikube
          ports:
            - containerPort: 8000
---
apiVersion: v1
```

```
kind: Service
metadata:
  name: api-service
spec:
  type: NodePort
  selector:
    app: api-rest
  ports:
    - protocol: TCP
      port: 8000      # Puerto expuesto en el Cluster K8s
      targetPort: 8000 # Puerto interno del contenedor FastAPI
      nodePort: 30080 # Puerto mapeado en nuestra maquina
      ↪ anfitriona
```

Fíjate en el uso de los tres guiones (--) en el archivo YAML; nos permiten definir múltiples objetos de Kubernetes (Deployment y Service) dentro de un mismo fichero de manera secuencial.

Finalmente, aplicamos el archivo YAML y comprobamos el acceso:

```
# Desplegar los recursos
kubectl apply -f api-k8s.yaml

# Comprobar que los Pods se están ejecutando
kubectl get pods

# Acceder a la API usando la IP de Minikube y el NodePort definido
curl http://$(minikube ip):30080/api/usuarios
```

5. Despliegue de una arquitectura de microservicios

El objetivo de esta práctica es aplicar de forma integral todos los conocimientos adquiridos sobre contenerización, orquestación y prácticas de seguridad (SecDevOps). Partiendo desde cero, los estudiantes deberán desarrollar, empaquetar mediante imágenes **Docker** y desplegar en un clúster de **Kubernetes** una arquitectura completa basada en microservicios para un sistema de gestión de compras. A diferencia de despliegues monolíticos tradicionales, esta aproximación exige gestionar la comunicación entre redes aisladas, la persistencia de datos y la configuración dinámica de variables críticas.

La arquitectura del sistema, ilustrada en la Figura 3, se compone de cuatro microservicios de desarrollo propio, apoyados por dos bases de datos relacionales distintas y un *broker* de mensajería para la comunicación asíncrona.

El punto de entrada a la infraestructura se materializa en el **API Gateway** (o Frontend Mock). Este componente actuará como la única interfaz expuesta al exterior del clúster de

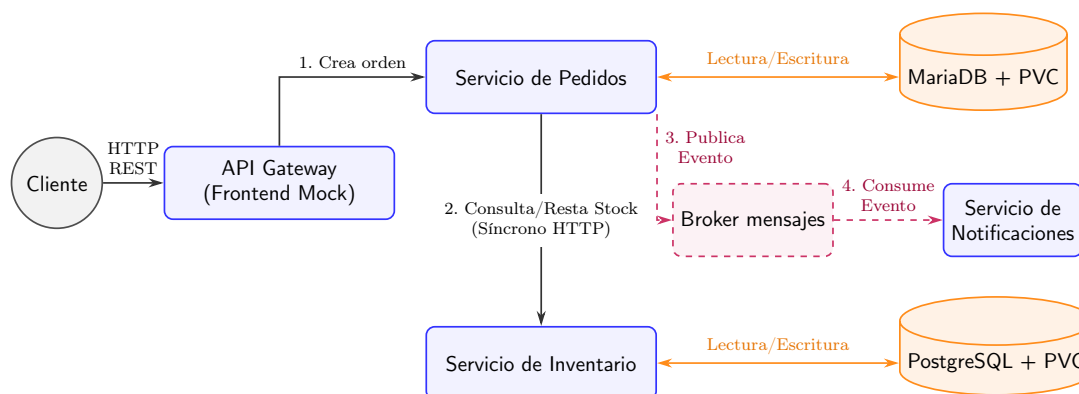


Figura 3: Arquitectura de microservicios propuesta para la práctica final.

Kubernetes, gestionada a través de un objeto **Ingress**. Su cometido principal es recibir las peticiones HTTP de los clientes (por ejemplo, una solicitud **POST** para realizar una compra) y enrutarlas internamente.

Desde la perspectiva de la seguridad, este componente introduce un mecanismo de control de acceso básico pero fundamental en el diseño de APIs: la *validación mediante un token estático*. Para que la petición sea aceptada, el cliente deberá incluir obligatoriamente en su solicitud una cabecera HTTP denominada **X-API-Key** (por ejemplo, **X-API-Key: mi_clave_secreta**). A nivel de desarrollo, el código del API Gateway interceptará las peticiones entrantes y leerá dicha cabecera. Si el valor coincide con el esperado, la petición continuará su curso; en caso contrario, se devolverá un código de error HTTP 401 (No autorizado). Siguiendo las directrices de SecDevOps, el valor esperado de esta clave **jamás debe codificarse directamente en el código fuente de la aplicación**. En su lugar, el valor real se almacenará de forma segura en un objeto **Secret** de **Kubernetes**. Durante el despliegue, el manifiesto del contenedor inyectará este **Secret** en forma de variable de entorno (por ejemplo, **API_TOKEN**), permitiendo que el código fuente simplemente compare la cabecera recibida con el contenido de dicha variable dinámica.

Una vez validadas, las solicitudes fluyen hacia el **Servicio de Pedidos**, que actúa como el orquestador central de la lógica de negocio. Este microservicio se encarga de iniciar el proceso de compra, guardando el registro de la transacción en una base de datos **MariaDB**. Para que los datos sobrevivan a posibles reinicios o fallos del contenedor de la base de datos, será necesario configurar un **PersistentVolumeClaim** asociado a su despliegue.

Durante el procesamiento de la compra, el *Servicio de Pedidos* debe verificar la disponibilidad del producto. Para ello, realizará una llamada síncrona mediante HTTP REST al **Servicio de Inventario**. La resolución de la dirección de red de este servicio se realizará de manera transparente gracias al servidor DNS integrado de **Kubernetes**, utilizando el nombre del objeto **Service** interno. A su vez, el *Servicio de Inventario* gestionará

sus propias existencias persistiendo la información en una base de datos independiente, en este caso **PostgreSQL**, la cual también deberá estar respaldada por su propio volumen persistente. Este aislamiento de bases de datos ejemplifica uno de los patrones fundamentales en las arquitecturas de microservicios: la descentralización de los datos.

Finalmente, si la transacción se realiza con éxito y el inventario es descontado, el sistema debe notificar al usuario. Para evitar bloquear el hilo de ejecución del *Servicio de Pedidos* esperando a que se envíe un correo electrónico, se empleará un enfoque asíncrono. El *Servicio de Pedidos* publicará un evento de “compra finalizada” en un **Broker de Mensajería**. De forma paralela, un **Servicio de Notificaciones** estará suscrito a dicha cola de mensajes; al detectar el nuevo evento, procederá a consumirlo y procesarlo. Por simplicidad, este procesamiento consistirá en imprimir un log por la salida estándar indicando que se ha enviado la notificación. Para el componente broker de mensajes se utilizará el código desarrollado en la práctica anterior.

6. Desarrollo y entrega final

El trabajo de esta práctica se estructurará a lo largo de las sesiones reservadas para esta práctica, requiriendo que cada equipo analice los requisitos, diseñe el software y redacte los archivos de despliegue. Como resultado final del laboratorio, los estudiantes deberán entregar un archivo comprimido estructurado que consolide todo el proyecto, acompañado de un breve documento arquitectónico.

El entregable final deberá contener exactamente los siguientes elementos:

- **Código fuente y empaquetado:** El código de los cuatro microservicios desarrollados, debidamente documentado con comentarios que faciliten la comprensión del sistema, junto con los ficheros «*Dockerfile*» correspondientes para su contenerización.
- **Orquestación e infraestructura:** Un directorio específico con todos los manifiestos YAML de Kubernetes necesarios para desplegar la plataforma. Esto incluye la instanciación de los **Deployments** (servicios y bases de datos), **Services** (enrutamiento interno), **PersistentVolumeClaims** (almacenamiento persistente), **Secrets** (credenciales seguras) e **Ingress** (exposición controlada).
- **Documentación de la arquitectura:** Un documento con una extensión máxima de 2 páginas (una hoja a doble cara) que incluya:
 - Vista de **módulos**.
 - Vista de **componentes y conectores**.
 - Vista de **distribución**.

Referencias

- [1] Install Docker Engine. [Online; <https://docs.docker.com/engine/install/>], 2026. Accedido el 27 de marzo de 2026.
- [2] Rootless mode. [Online; <https://docs.docker.com/engine/security/rootless/>], 2026. Accedido el 27 de marzo de 2026.
- [3] Docker Volumes. [Online; <https://docs.docker.com/engine/storage/volumes/>], 2026. Accedido el 27 de marzo de 2026.
- [4] Networking. [Online; <https://docs.docker.com/engine/network/>], 2026. Accedido el 27 de marzo de 2026.
- [5] Docker Hub Container Image Library. [Online; <https://hub.docker.com/>], 2026. Accedido el 27 de marzo de 2026.
- [6] Minikube start. [Online; <https://minikube.sigs.k8s.io/docs/start/>], 2026. Accedido el 27 de marzo de 2026.
- [7] Kubernetes Components. [Online; <https://kubernetes.io/docs/concepts/overview/components/>], 2026. Accedido el 27 de marzo de 2026.
- [8] Encrypting Confidential Data at Rest. [Online; <https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>], 2026. Accedido el 27 de marzo de 2026.